

Оглавление

[Задания на лабораторные работы по курсу АВМиС\(АПЭВМ\). 2](#)

[Лабораторная работа №1 " Последовательный порт " 2](#)

[ЗАДАНИЕ 2](#)

[Теоретические сведения 2](#)

[Вопросы к защите 3](#)

[Список рекомендуемой литературы 3](#)

[Лабораторная работа №2. Программирование контроллера прерываний. 3](#)

[Задание 3](#)

[Теоретические сведения 3](#)

[ВАРИАНТЫ заданий 4](#)

[Вопросы к защите 4](#)

[Список рекомендуемой литературы 5](#)

[Лабораторная работа №3. Программирование системного таймера. 5](#)

[Задание 5](#)

[Теоретические сведения 5](#)

[ВАРИАНТЫ заданий 6](#)

[Вопросы к защите 7](#)

[Список рекомендуемой литературы 7](#)

[Лабораторная работа № 4. Программирование часов реального времени. 7](#)

[Задание 7](#)

[Теоретические сведения 8](#)

[Вопросы к защите 9](#)

[Список рекомендуемой литературы 9](#)

[Лабораторная работа № 5. Программирование клавиатуры. 10](#)

[Задание 10](#)

[Теоретические сведения 10](#)

[Вопросы к защите 11](#)

[Список рекомендуемой литературы 11](#)

[Лабораторная работа N°6 11](#)

[Защищенный и реальный режим процессора. Переход из одного режима в другой и обработка прерываний. 11](#)

[Задание 11](#)

[Лабораторная работа N°7. Защищенный режим процессора. Мультизадачность. 12](#)

[Задание 12](#)

Задания на лабораторные работы по курсу АВМ и С(АПЭВМ).

Условия выполнения лабораторных работ:

1. Программирование всех устройств осуществляется через порты ввода/вывода.
2. Любое задание на лабораторную работу может включать в себя набор некоторых реализаций которых *обязательна* для выполнения лабораторной работы.
3. Операционная система выполнения лабораторной работы – любая (рекомендуется Windows).
4. Язык программирования – любой (рекомендуется C, C++ , ASM).
5. Для допуска к экзамену необходимо выполнить 7 ЛР или 7,8.
6. На выполнение и защиту ЛР 1-6 отводится 4 часа. На ЛР7 – 8 часов.
7. Если ЛР сдана в срок, то оценка идет с коэффициентом 1, если просрочка равна (например, на выполнение ЛР1 затрачено более 4 часов и сдается она вместе с ЛР2), то имеет коэффициент $\frac{1}{2}$, если просрочка равна двум ЛР, то предыдущая имеет коэффициент

Лабораторная работа №1 " Последовательный порт "

ЗАДАНИЕ

Разработать программный модуль реализации процедуры передачи (приема) информации через последовательный интерфейс.

Программа должна демонстрировать программное взаимодействие с последовательным интерфейсом с использованием следующих механизмов:

1. прямое взаимодействие с портами ввода-вывода (write, read)
2. использование BIOS прерывания 14h,
3. работа с COM-портом через регистры как с устройством ввода-вывода.

Теоретические сведения

Последовательный порт или COM-порт - двунаправленный последовательный интерфейс, предназначенный для обмена байтовой информацией. Последовательный порт отличается от параллельного порта тем, что информация через него передаётся по одному биту, бит за битом (в отличие от параллельного порта). Наиболее часто для последовательного порта персональных компьютеров используется стандарт RS-232C.

Простейший алгоритм передачи данных через последовательный интерфейс состоит из следующих фаз:

- инициализация порта;
- передача данных;
- прием данных;
- анализ состояния порта.

Вопросы к защите

1. Назначение сигналов COM порта по стандарту RS-232C.
2. Назовите регистры последовательных портов и их назначение.
3. Алгоритмы режимов синхронизации приема- передачи данных
4. Для чего используется режим диагностики и как включается?
5. Назовите преимущества последовательных портов.

Список рекомендуемой литературы

1. Зубков С.В. Ассемблер для DOS, Windows и Unix.
2. Пирогов В.Ю. Assembler. Учебный курс.
3. <https://www.softelectro.ru/rs232prog.html>

Лабораторная работа №2. Программирование контроллера прерываний.

Задание

Написать резидентную программу выполняющую перенос всех векторов аппаратных прерываний ведомого контроллера на пользовательские прерывания. При этом необходимо написать аппаратных прерываний, которые будут установлены на используемые пользовательские прерывания выполнять следующие функции:

1. Выводить на экран в двоичной форме следующие регистры контроллеров прерывания (как и ведомого):
 - регистр запросов на прерывания;
 - регистр обслуживаемых прерываний;
 - регистр масок.

При этом значения регистров должны выводиться всегда в одно и то же место экрана.

2. Осуществлять переход на стандартные обработчики аппаратных прерываний, для нормальной работы компьютера.

Теоретические сведения

Каждый момент времени центральный процессор может работать только с одним устройством. Циклический опрос каждого устройства с последующей обработкой запроса является неэффективным. Решение задачи оказалось контроллер прерываний, который принимает запросы от устройств и в соответствии с приоритетом направляет их процессору, если прерывание устройства не замаскировано (разрешено) в регистре масок. Если прерывание разрешено, то устройство его запросило, то устанавливается соответствующий устройству бит в регистре запросов.

Контроллер прерываний состоит из двух микросхем, подключенных каскадно (ведущий и ведомый контроллеры), каждая из которых имеет по 8 линий прерываний (IRQ0-IRQ7, IRQ8-IRQ15). Каждая линия закреплено определенное устройство.

Когда процессор получает запрос на прерывание, он сохраняет свое текущее состояние и переключается на выполнение запрошенной операции. При этом устанавливается бит обслуживания (бит запроса сбрасывается). После обслуживания прерывания сбрасывается бит обслуживания, посылается сигнал EOI (end of interrupt), процессор переключается на выполнение следующей операции.

ранее задачу.

Для доступа к контроллеру прерываний используются порты 20h и 21h (для ведущего), и ведомого).

Регистр масок доступен через порт 21h /A1h. Чтобы изменить определенный бит, ну значение из этого регистра, изменить нужный бит, записать значение обратно.

Чтобы считать регистр запросов, его нужно выбрать (записать в 20h/A0h значение 0 считать содержимое из порта 20h/A0h.

Чтобы считать регистр обслуживания, его нужно выбрать (записать в 20h/A0h значение считать содержимое из порта 20h/A0h.

Для резидентной программы понадобится следующий фрагмент кода:

```
unsigned far *fp; //объявляем указатель
```

```
FP_SEG (fp) = _psp; // получаем сегмент
```

```
FP_OFF (fp) = 0x2c; // и смещение сегмента данных с переменными среды
```

```
_dos_freemem(*fp); //чтобы его освободить
```

```
_dos_keep(0,(_DS - _CS)+(_SP/16)+1); //оставляем резидентной, указывая //первым па  
код завершения, а //вторым - объем памяти, который должен //быть зарезервирован;  
программы //после ее завершения
```

ВАРИАНТЫ заданий

Номер варианта	1	2	3	4	5	6	7	8
Базовый вектор	60H / 08H	68H / 08H	70H / 08H	78H / 08H	80H / 08H	88H / 08H	90H / 08H	98H / 08H
Номер варианта	11	12	13	14	15	16	17	18
Базовый вектор	B0H / 08H	B8H / 08H	C0H / 08H	C8H / 08H	D0H / 08H	D8H / 08H	08H / 60H	08H / 68H

Номер варианта	21	22	23	24	25	26	27	28
Базовый вектор	08H / 80H	08H / 88H	08H / 90H	08H / 98H	08H / A0H	08H / A8H	08H / B0H	08H / B8H

Вопросы к защите

1. Для чего в программе обработчика прерываний необходимо указывать команду EO
2. Для чего используются команды CLI и STI?
3. Объяснить понятие «вектор прерывания».
4. Что делает команда IRET?
5. Что такое «вложенное прерывание»?
6. Назначение таблицы векторов прерываний.

Список рекомендуемой литературы

4. В. Несвижский "Программирование аппаратных средств в Windows", с. 495 (486).

Лабораторная работа №3. Программирование системного таймера.

Задание

Задание состоит из двух частей. Первая часть общая для всех. Вторая часть по вариантам

Первая часть (общее задание). Запрограммировать второй канал таймера таким образом, чтобы компьютер издавал звуки.

Вторая часть:

1. Для всех каналов таймера считать слово состояния и вывести его на экран в двоичной форме
2. Для всех каналов таймера рассчитать коэффициент деления (значение счетчика CE) и вывести его на экран в шестнадцатеричной форме.

Теоретические сведения

Системный таймер - устройство, подключенное к линии запроса IRQ0 и вырабатывающее interrupt 8h. Таймер реализуется на микросхеме Intel 8253 или 8254 (Отечественные аналоги: К1810ВИ54).

Таймер состоит из трёх независимых каналов: 0 - отсчитывает текущее время после включения компьютера, 1- для работы с контроллером прямого доступа к памяти, 2 - связан с динамиком.

Каждый канал содержит регистры:

- состояния канала RS (8 разрядов);
- управляющего слова RSW (8 разрядов);
- буферный регистр OL (16 разрядов);
- регистр счетчика CE (16 разрядов);
- регистр констант пересчета CR (16 разрядов)

Каналы таймера подключаются к внешним устройствам при помощи линий:

- GATE - управляющий вход;
- CLOCK - вход тактовой частоты;
- OUT - выход таймера.

Регистр счетчика CE работает в режиме вычитания. Его содержимое уменьшается на единицу при каждом тактовом импульсе сигнала CLOCK при условии, что на вход GATE установлен уровень логической 1.

В зависимости от режима работы таймера при достижении счетчиком CE нуля тактовым импульсом изменяется выходной сигнал OUT.

Буферный регистр OL предназначен для запоминания текущего содержимого счетчика CE без остановки процесса счета. После запоминания буферный регистр доступен для чтения.

Регистр констант пересчета CR может загружаться в регистр счетчика, если это происходит в текущем режиме работы таймера.

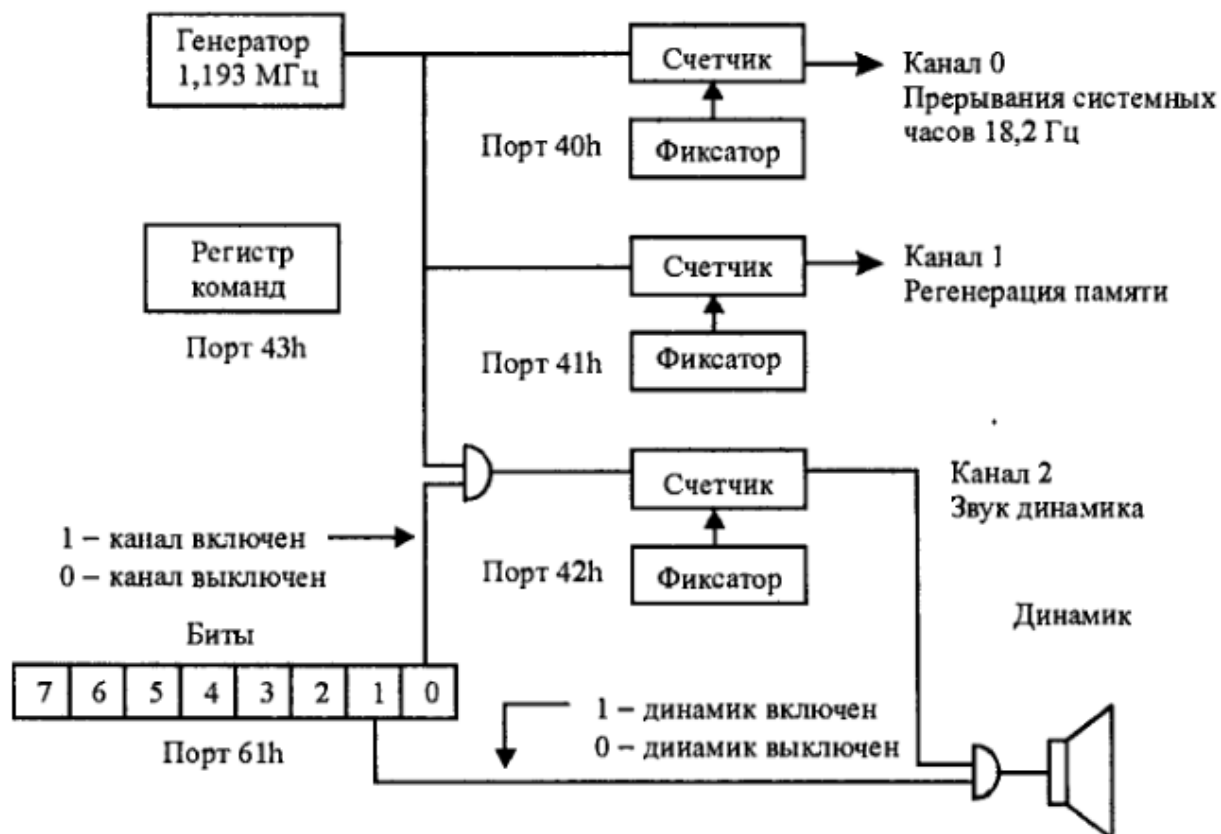
Возможны шесть режимов работы таймера, которые делятся на три типа:

Режимы 0, 4 - однократное выполнение функций.

Режимы 1, 5 - работа с перезапуском.

Режимы 2, 3 - работа с автозагрузкой.

Для подключения динамика используется порт 61h.



ВАРИАНТЫ заданий

Вариант	Частота, Гц (длительность, мс)									
1.	659 (200)	622 (200)	659 (200)	622 (200)	659 (200)	493 (200)	622 (200)	523 (200)	440 (200)	
2.	293 (200)	392 (400)	293 (200)	392 (400)	523 (200)	659 (200)	880 (400)	784 (200)	698 (200)	

3.	784 (200)	392 (200)	440 (200)	440 (200)	784 (200)	440 (200)	493 (200)	493 (200)	
4.	329 (200)	329 (100)	329 (200)	415 (400)	523 (200)	659 (200)	587 (400)	523 (200)	659 (400)
5.	196 (400)	261 (400)	329 (400)	196 (400)	261 (400)	329 (400)	196 (400)	261 (400)	329 (400)
6.	392 (400)	392 (400)	392 (400)	329 (1000)	349 (400)	349 (400)	349 (400)	293 (800)	293 (1000)
7.	392 (400)	329 (400)	329 (400)	392 (400)	329 (400)	329 (400)	392 (400)	349 (400)	329 (400)
8.	329 (400)	246 (400)	246 (400)	261 (400)	293 (400)	329 (400)	392 (400)	349 (400)	
9.	392 (800)	392 (400)	293 (400)	196 (400)	196 (400)	392 (400)	392 (400)	293 (400)	196 (400)
10.	349 (600)	392 (300)	440 (600)	349 (300)	440 (300)	440 (300)	392 (300)	349 (300)	392 (300)

Вопросы к защите

1. Перечислите режимы работы системного таймера и их типы. Объясните, в чём различия.
2. Объясните назначение и использование каналов таймера в IBM PC.
3. Форматы управляющего слова для задания режима работы таймера.
4. Последовательность программирования системного таймера.

5. Как системный таймер используется для генерации звуков?
6. Какую информацию можно получить из слова состояния?
7. В каких пределах может быть частота звука, генерируемого с помощью таймера?
8. *Нарисуйте временные диаграммы работы таймера для 2 и 3 режимов.

Список рекомендуемой литературы

В. Несвижский "Программирование аппаратных средств в Windows", с. 317(311), 33

Лабораторная работа № 4. Программирование часов реального времени.

Задание

Задание по лабораторной работе состоит из двух частей. Первая часть общая для всех, вторая – выдаваемым преподавателем.

Первая часть (общее задание). Написать программу, которая будет считывать и устанавливать реального времени. Считанное время должно выводиться на экран в удобочитаемой форме.

Условия выполнения лабораторной работы для первой части:

1. Перед началом установки новых значений времени необходимо считывать и анализировать регистра состояния 1 на предмет доступности значений для чтения и записи. Начинать операцию установки новых значений, можно только в случае, когда этот бит установлен в '0' – то есть, регистры доступны.
2. После проверки доступности регистров (пункт 1), необходимо отключить внутренний делитель часов реального времени, воспользовавшись старшим битом регистра состояния 2. После операции установки значений этот бит должен быть сброшен, для возобновления внутреннего обновления часов реального времени.
3. Новые значения для регистров часов реального времени должны вводиться с клавиатуры и выводиться на экран пользователя в виде.

Вторая часть.

1. Используя аппаратное прерывание часов реального времени и режим генерации и прерываний реализовать функцию задержки с точностью в миллисекунды (с возможностью частоты генерации).

Условия выполнения данного варианта:

1. Задержка должна вводиться с клавиатуры в миллисекундах в удобной для пользователя
 2. После окончания периода задержки программа должна сообщить об этом в любой форме, зависание компьютера не допускается.
 3. Предусмотреть возможность изменения частоты генерации периодических прерываний клавиатуры соответствующих значений
2. Используя аппаратное прерывание часов реального времени и режим будильника ЧРВ (4А не желательно) реализовать функции программируемого будильника.
 1. Время будильника вводиться с клавиатуры в удобной для пользователя форме.
 2. При срабатывании будильника программа должна сообщить об этом в любой форме, зависание компьютера не допускается.

Общие условия для всей лабораторной работы.

1. В программе должно быть реализовано меню, позволяющее выбрать тестируемые функции: времени, считывание времени, задержка и т.д.)
2. Весь ввод/вывод данных с/на консоль должен выполняться в удобной для пользователя форме
3. Зависание компьютера не допускается ни в ходе работы программы, ни после ее завершения

Теоретические сведения

Часы реального времени (Real Time Clock) - специальный модуль, используемый для непрерывного отсчёта времени. За счёт использования батарейки этот модуль работает даже когда компьютер выключен.

Для хранения данных в часах реального времени предусмотрено несколько регистров, записанных в формате BCD):

Адрес регистра	Описание значения
00h	Текущая секунда

	Текущая секунда
01h	Секунды будильника
02h	Текущая минута
03h	Минуты будильника
04h	Текущий час
05h	Часы будильника
06h	Текущий день недели (01-07, 01 - воскресенье)
07h	Текущий день месяца
08h	Текущий месяц
09h	Текущий год (последние 2 цифры)



Для доступа к часам реального времени используются всего два порта: 70h и 71h. Порт 70h испол для записи и позволяет выбрать адрес регистра в CMOS памяти. Порт 71h применяется как для за чтения данных из указанного (через порт 70h) регистра CMOS. Оба порта являются 8-разрядным бит 7 в порту 70h не относится к работе с RTC(ЧРВ), а управляет режимом немаскируемых прерывания запрещены). Если у вас будут проблемы с доступом к регистрам CMOS, следу прерывания (команда cli) перед началом работы и разрешать прерывания (команда sti) после. Но этого не требуется.

Основное назначение часов - это непрерывный отсчет времени суток даже в то когда компьютер выключен, а также извещение процессора о наступлении определенного времени (режим будильника). Часы реального времени реализс базе микросхемы типа Motorola MC 146818 (RT/CMOS RAM Chip) и обеспечи непрерывный отсчет времени при выключении питания на компьютере за счет аккумуляторной батареи. Эта микросхема включает также память, содержи сохраняется при исключении питания. Аппаратура часов реального времени и только первые 14 байт этой памяти, а остальные предназначены для сохранени конфигурации компьютера. Часы позволяют генерировать три типа прерывани линии 0 второго контроллера прерываний (IRQ 8), что соответствует в IBM PC прерыванию 70h:

1. периодические прерывания с интервалом 976.562 мкс;
2. прерывание от будильника;
3. прерывание по окончании обновления значения часов.

Регистры часов

Как уже говорилось ранее, часы используют первые 14 байт памяти микросхем Motorola MC146818, которые будем в дальнейшем называть регистрами часов считать любой памяти микросхемы, в том числе и данные из регистра часов н

выполнить следующие шаги:

1. записать адрес считываемого регистра в порт 70h;
2. считать данные регистра из порта 71h.

Например, следующие команды используются для чтения регистра 0 часов реального времени:

```
mov al,0 ; Записать адрес регистра
out 70h,al
jmp short $+2
in al,71h ; Считать регистр
```

Аналогично, для записи значения в регистр часов требуется занести адрес модифицируемого регистра в порт 70h, после чего вывести в порт 71h записываемое значение. Например:

```
mov al,0 ; Записать адрес регистра
out 70h,al
jmp short $+2
mov al,20h ; Занести новое значение
out 71h,al
```

Следует отметить, что чтение и запись данных порта 71h должны выполняться немедленно после записи номера регистра в порт 70h. Кроме того, в процессе ввода-вывода с регистрами часов должны быть запрещены прерывания, что позволяет избежать модификации порта 70h процедурой обработки прерывания до чтения записи данных.

Адресация регистров часов приведена в таблице:

Адрес регистра	Назначение регистра
00h	Секунды

01h	Секунды для будильника
02h	Минуты
03h	Минуты для будильника
04h	Часы
05h	Часы для будильника
06h	День недели
07h	Дата
08h	Месяц
09h	Год
0Ah	Регистр состояния 1
0Bh	Регистр состояния 2
0Ch	Регистр состояния 3
0Dh	Регистр состояния 4

32h	Столетие
-----	----------

Регистры времени 00h-09h и 32h содержат время в двоично-десятичном формате (например, дата 26 хранится в виде 26h). Столетие задается в виде первых двух полных номеров года (например, 19h). На машинах PS/2 байт столетия расположен по адресу 37h.

Регистр состояния 1 (A) имеет следующий формат:

7	6	5	4	3	2	1	0
UIP	DV	RS					

Бит *UIP* определяет момент обновления показаний часов и может быть установлен, если часы доступны для чтения или - 1 - аппаратура выполняет обновление показаний. Для правильного чтения или записи значений часов эти операции необходимо выполнять, когда значение бита *UIP* равно 0.

Биты *DV* задают значение частоты для обновления показаний часов. Для правильного функционирования часов данные биты должны быть установлены в значение, соответствующее частоте 32.768 КГц. Это единственная частота, обеспечивающая правильное время в часах.

Биты *RS* позволяют выбрать делитель выходной частоты. Для правильного функционирования эти биты должны быть установлены в значение 0110, что соответствует выходной частоте прямоугольных колебаний 1.024 КГц и интервалу времени между периодическими прерываниями 976.562 мкс.

Формат **регистра состояния 2 (B)** приведен ниже:

7	6	5	4	3	2	1	0
UP D	PI E	AI E	UI E	SQW E	D M	C F	DS E

Бит *UPD* используется для обновления показания часов. Когда он устанавливается, внутренний цикл обновления часов прекращается, и программа может



В этом регистре состояния задействован только бит *VRB* который сигнализирует о состоянии аккумуляторной батареи. Единичное значение говорит о нормальном питании. Нулевое значение указывает, что батарея разряжена и данные в часах недостоверны.

Вопросы к защите

Перечислите режимы работы системного таймера и их типы. Объясните, в чём различия.

Перечислите регистры системного таймера и их назначение.

Список рекомендуемой литературы

В. Несвижский "Программирование аппаратных средств в Windows", с. 323(316).

Лабораторная работа № 5. Программирование клавиатуры.

Задание

Программируя клавиатуру, помогать ее индикаторами. Алгоритм мигания произвольный. Вспомогательные функции:

программируя клавиатуру посылать ее индикаторами. Формат маски произвольный. Основы программы, необходимые для выполнения лабораторной работы:

1. Запись байтов команды должна выполняться только после проверки незанятости входного контроллера клавиатуры. Проверка осуществляется считыванием и анализом регистра контроллера клавиатуры.
2. Для каждого байта команды необходимо считывать и анализировать код возврата. В случае кода возврата, требующего повторить передачу байта, необходимо повторно, при необходимости несколько раз, выполнить передачу байта. При этом повторная передача данных не исключает всех оставшихся условий.
3. Для определения момента получения кода возврата необходимо использовать аппаратное устройство клавиатуры.
4. Все коды возврата должны быть выведены на экран в шестнадцатеричной форме.

Теоретические сведения

Для работы с клавиатурой используется 2 регистра: 60h – регистр данных, 64h – регистр статуса).

Для управления индикаторами через 60h отправляется код EDh. Затем маска, в которой должны загореться индикаторы.

Формат «маски»:

Биты 7-3 не используются	Caps Lock	Num Lock	Scroll Lock
--------------------------	-----------	----------	-------------

1 – включить

0 - выключить

Так клавиатура работает медленно, запись байтов команды должна выполняться только после проверки незанятости входного регистра контроллера клавиатуры. Проверка осуществляется считыванием и анализом регистра состояния контроллера клавиатуры (64h, бит 1).

На обработку каждого байта клавиатура отвечает кодом возврата. Если в регистре 60h на

FA – байт обработан успешно,

FE – произошла ошибка.

В случае ошибки передачу байта нужно повторить. Пересылка выполняется до 3 раз, если исчезла, нужно вывести сообщение и выйти из программы.

При нажатии клавиши блок клавиатуры передает ее код сканирования центральному процессору. Когда клавиша отпускается, клавиатура снова передает ее код, но увеличенный в 16 раз (т.е. шестнадцатеричное значение 80). То есть, коды для нажатия и отпускания клавиш различны.

Когда выполняется какое-либо действие с клавишей (нажатие или отпускание), контроллер клавиатуры обнаруживает его и запоминает в буфере. Затем формируется прерывание на линии IRQ 9.

Принципы работы клавиатуры

Клавиатура выполнена, как правило, в виде отдельного устройства, подключаемого к компьютеру тонким кабелем. Малогабаритные компьютеры типа Lap-Тор используют встроенную клавиатуру.

Внутри клавиатуры находится специализированная микросхема (контроллер клавиатуры), которая отслеживает нажатия на клавиши и посылает код нажатой клавиши в персональный компьютер.

Если рассмотреть сильно упрощенную принципиальную схему клавиатуры, представленную на рисунке, можно заметить, что все клавиши находятся в матрице:

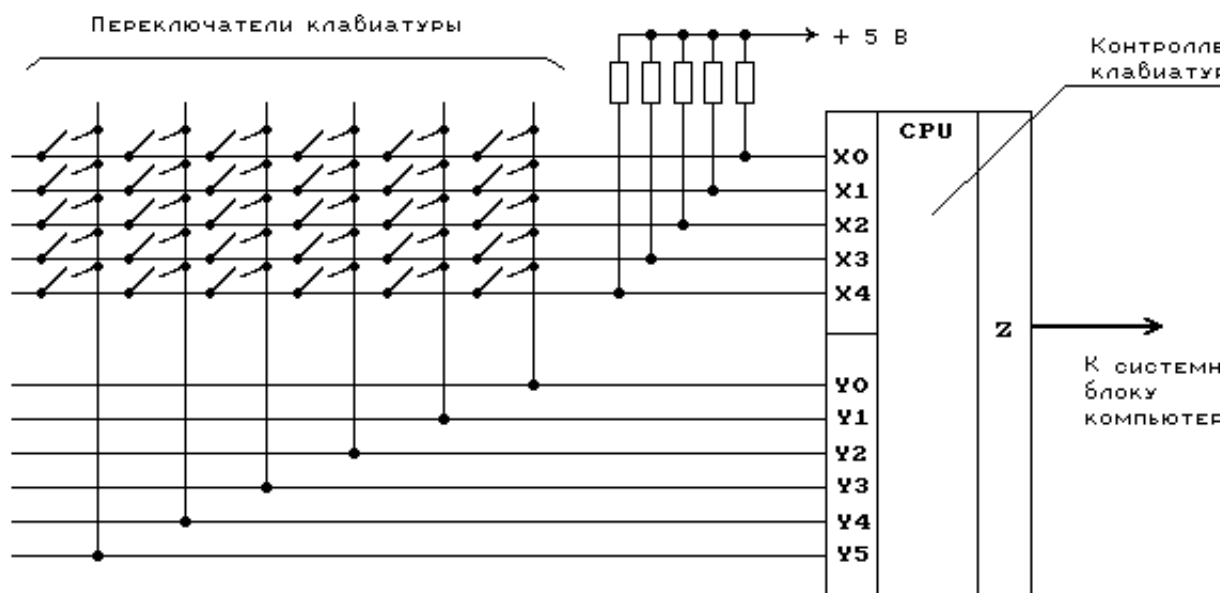


Рис.1. Упрощенная схема клавиатуры

Все горизонтальные линии матрицы подключены через резисторы к

питания +5 В. Клавиатурный компьютер имеет два порта - выходной и входной. Входной порт подключен к горизонтальным линиям матрицы (Y0-Y5), а выходной - к вертикальным (X0-X5).

Устанавливая по очереди на каждой из вертикальных линий напряжения, соответствующий логическому 0, клавиатурный контроллер опрашивает состояние горизонтальных линий. Если ни одна клавиша не нажата, то уровень напряжения на всех горизонтальных линиях соответствует логическому 1 (т.к. все эти линии подключены к источнику питания +5 В через резисторы).

Если оператор нажмет на какую-либо клавишу, то соответствующая вертикальная и горизонтальная линии окажутся замкнутыми. Когда на определенной вертикальной линии контроллер установит значение логического 0, то на соответствующей горизонтальной линии также будет соответствующее значение логического 0.

Как только на одной из горизонтальных линий появится уровень логического 0, клавиатурный контроллер фиксирует нажатие на клавишу. Он посылает персональному компьютеру запрос на прерывание и номер клавиши в виде байта. Аналогичные действия выполняются и тогда, когда оператор отпускает клавишу.

Номер клавиши, посылаемый клавиатурным процессором, однозначно соответствует раскладке клавиатурной матрицы и не зависит напрямую от обозначения, нанесенных на поверхность клавиш. Этот номер называется скан-кодом (Scan Code).

Слово scan ("сканирование"), подчеркивает тот факт, что клавиатурный контроллер сканирует клавиатуру для поиска нажатой клавиши.

Но программе нужен не порядковый номер нажатой клавиши, а соответствующий обозначению на этой клавише ASCII-код. Этот код не зависит однозначно от скан-кода, т.к. одной и той же клавише могут соответствовать несколько значений ASCII-кода. Это зависит от состояния других клавиш. Например, клавиша с обозначением '1' используется еще и для ввода символа '2' (если она нажата вместе с клавишей SHIFT).

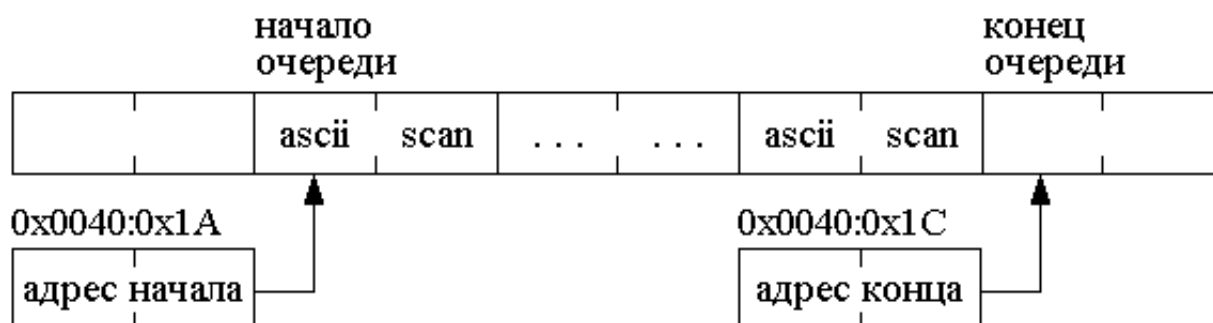
Поэтому все преобразования скан-кода в ASCII-код выполняются программно, обеспечиваемыми внутри ПК. Как правило, эти преобразования выполняются BIOS. Для использования символов кириллицы эти модули реализованы в клавиатурных драйверах.

Если нажать на клавишу и не отпускать ее, клавиатура перейдет автоповтора. В этом режиме в центральный компьютер автоматически некоторый период времени, называемый периодом автоповтора, посылает код нажатой клавиши. Режим автоповтора облегчает ввод с клавиатуры большого количества одинаковых символов.

Структура буфера клавиатуры

Клавиатура является основным средством ввода информации при работе с компьютером. В компьютерах IBM PC взаимодействие программ с клавиатурой организовано в асинхронном режиме. При нажатии клавиши на клавиатуре код нажатой клавиши поступает в буфер клавиатуры и хранится там до тех пор, пока программа не затребует его. Если скорость нажатия клавиш и скорость обработки нажатия клавиш программой отличаются, то может возникнуть ситуация, когда ожидается поступления кода очередной клавиши в буфер, или коды клавиш накапливаются в буфере клавиатуры. Хранение информации организовано в виде очереди (FIFO - First Input First Output), буфер находится в оперативной памяти компьютера и доступен для обычной программы чтения и записи.

Очередь буфера клавиатуры реализована следующим образом:



Каждой нажатой клавише в очереди соответствует ячейка размером 2 байта (1 байт для кода ASCII и 1 байт для кода скана).

байта, в первом байте записан ascii код, а во втором - scan код нажатой клавиши. Следует учитывать, что при считывании слова из буфера клавиатуры получается в старшем байте, а ascii код в младшем байте слова в соответствии с порядком следования байтов принятом для IBM PC. В двух словах операционная система по адресам 0x0040:0x1A и 0x0040:0x1C хранятся смещения начала (головы) и конца (хвоста) очереди буфера клавиатуры. Следует обратить внимание, что эти смещения указаны относительно адреса 0040:0000h. Буфер размером 32 байта (16 слов) расположен в компьютере IBM PC адресу 0000h:041Eh и организован в виде кольцевой очереди. Следует отметить, что рост буфера (клавиши нажимаются, а символы из буфера не считываются) происходит в сторону хвоста.

В компьютерах моделей IBM PC/AT и IBM PS/2 расположение клавиатурного буфера задается содержимым двух слов памяти с адресами 0000h:0040h (смещение адреса начала буфера относительно 0040h) и 0000h:0482h (смещение адреса конца буфера относительно 0040h). Обычно эти ячейки памяти содержат значения, соответственно, 001Eh и 003Eh, что соответствует расположению клавиатурного буфера в IBM PC/AT и IBM PS/2 его расположению в IBM PC.

Указателями на начало и конец клавиатурного буфера обычно являются обработчики прерываний INT 09h и INT 16h. Программа извлекает из буфера коды нажатых клавиш, используя различные функции прерывания INT 09h и INT 16h.

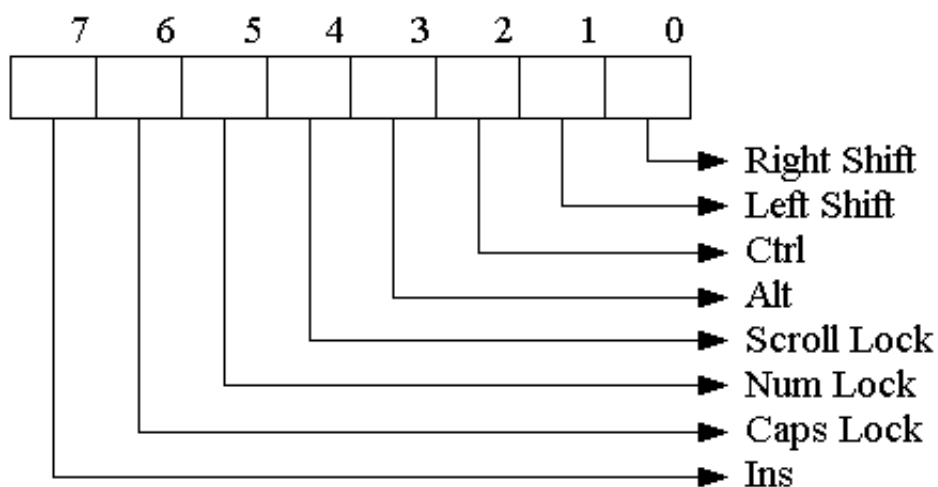
Заметим, что вы можете удалить все символы из буфера клавиатуры, сбросив оба указателя на начало буфера. Однако есть более предпочтительный способ сброса буфера с использованием, например, прерывания BIOS INT 16h.

Для чтения scan-кода последней нажатой клавиши (даже в том случае, если буфер уже заполнен) можно использовать 60-й порт. При этом следует учитывать, что многократное обращение к порту будет возвращать с тем же результатом до тех пор, пока не будет нажата следующая клавиша на клавиатуре.

При переполнении внутреннего буфера клавиатуры или буфера, расположенного в области данных BIOS программа-обработчик прерываний INT 09h генерирует звуковой сигнал.

Кроме буфера клавиатуры и ячеек памяти с указателями головы и хвоста очереди с обработкой нажатия клавиш клавиатуры связано ещё несколько элементов в области памяти данных BIOS. Одним из этих элементов является флаг состояния клавиатуры, располагающийся в области переменных BIOS по адресу 0000:0417h. Его размер составляет один байт, отдельные биты

содержат информацию о состоянии клавиш Shift, Ctrl, Alt, Scroll Lock, Caps Lock и Ins. Соответствие перечисленных клавиш и битов флага по рисунку:



Вопросы к защите

Что такое код возврата?

Как определить, является ли код кодом нажатия клавиши или ее отпускания?

Через какие порты происходит доступ к клавиатуре?

Список рекомендуемой литературы

В. Несвижский "Программирование аппаратных средств в Windows"

Лабораторная работа №6

Защищенный и реальный режим процессора
Переход из одного режима в другой и обратный

прерывании.

Задание

Написать программу, которая выполняет следующие действия:

Переход из реального режима в защищенный.

Перехватывает аппаратное прерывание от клавиатуры и заданное прерывание, в обработчике которого выполняет определенные действия. И от клавиатуры реализуют все, номер аппаратного прерывания задается по (два варианта).

Прерывание от таймера.

Прерывание от часов реального времени.

По наступлению определенного события выполняет обратный переход из защищенного режима в реальный и завершает свою работу.

Для прерываний от таймера и часов реального времени обработчик должен отслеживать количество вызовов прерывания и отсчитывать секунды на экран. Количество секунд после которых выполняется обратный переход в реальный режим и выход из программы (то самое определенное считывается с клавиатуры перед переходом в защищенный режим.

Для прерывания от клавиатуры необходимо считывать скан-коды клавиш на экран. По нажатию определенной клавиши (любой на выбор) осуществляется обратный переход в реальный режим и выход из программы.

При выполнении данной лабораторной работы должны быть соблюдены условия:

После завершения работы программы компьютер должен продолжать функционировать. Зависания, перезагрузки и другие аналогичные недопустимы.

Переход в защищенный режим процессора должен быть выполнен по используемому в процессорах начиная с 386. Переход в защищенный режим с использованием алгоритма для 286 процессора недопустим.

Лабораторная работа №7. Защищенный режим процессора. Мультизадачность.

Задание

Написать программу, которая выполняет следующие действия:

Выполняет переход из реального режима в защищенный.

Организует минимум три задачи, переключающиеся между собой.

Выполняет обратный переход из защищенного режима в реальный.

Каждая задача должна выполнять действие, по которому можно будет сделать вывод, что она работает (например, одна задача выводит символы справа налево, другая организует счетчик, третья выводит символы слева направо).

Возврат в реальный режим осуществляется по нажатию на “Esc”.

Переключение задач должно выполняться с использованием аппаратных прерываний по вариантам:

Прерывание от таймера.

Прерывание от часов реального времени.

Для прерываний от таймера и часов реального времени переключение осуществляется через определенные промежутки времени, вводимые с клавиатуры при переходе в защищенный режим.

При выполнении данной лабораторной работы должны быть выполнены следующие условия:

После завершения работы программы компьютер должен продолжать

функционировать. Зависания, перезагрузки и другие аналогичные недопустимы.

Переход в защищенный режим процессора должен быть выполнен по используемому в процессорах начиная с 386. Переход в защищенный использованием алгоритма для 286 процессора недопустим.
